Compilador C- (C Minus)

Implementação de um compilador para a linguagem didática C-, incluindo analisador léxico (scanner), analisador sintático (parser) e construção da Árvore Sintática Abstrata (AST).

📋 Descrição

Este projeto implementa as fases iniciais de um compilador para a linguagem C-, conforme especificado no livro "Compiler Construction: Principles and Practice" de Kenneth C. Louden.

Componentes Implementados

- ✅ **Scanner (Analisador Léxico)**: Reconhece tokens, identificadores, palavras-chave, números e operadores
- ✅ **Tabela de Símbolos**: Gerencia identificadores com informação de tipo, escopo e categoria
- ✅ **Parser (Analisador Sintático)**: Analisa a estrutura gramatical do código
- ✅ **AST (Árvore Sintática Abstrata)**: Representação estrutural do programa
- ✅ **Tratamento de Erros**: Mensagens de erro léxicos e sintáticos com número de linha

🛠️ Tecnologias Utilizadas

- **Linguagem**: C
- **Flex**: Gerador de analisador léxico
- **Bison**: Gerador de analisador sintático
- **GCC**: Compilador C

📦 Estrutura do Projeto

```
.
├── scanner.l      # Especificação do analisador léxico (Flex)
├── parser.y       # Especificação do analisador sintático (Bison)
├── ast.h          # Definições da Árvore Sintática Abstrata
├── ast.c          # Implementação das funções da AST
├── symtab.h       # Definições da Tabela de Símbolos
├── symtab.c       # Implementação da Tabela de Símbolos
├── build_symtab.c # Construção da tabela a partir da AST
├── common.h       # Definições compartilhadas
├── common.c       # Funções auxiliares
├── main.c         # Programa principal
├── Makefile       # Script de compilação
└── README.md      # Este arquivo
```

🚀 Como Compilar

Pré-requisitos

- GCC (GNU Compiler Collection)
- Flex (Fast Lexical Analyzer)
- Bison (GNU Parser Generator)

Instalação das Dependências (Ubuntu/Debian)

```bash
sudo apt-get update
sudo apt-get install build-essential flex bison
```

Compilação

```bash
make
```

Ou para limpar e recompilar:

```bash
make rebuild
```

📝 Como Usar

Executar o Compilador

```bash
./cminusc <arquivo.cm>
```

Exemplo:

```bash
./cminusc test.cm
```

Saída

O compilador produz:

1. Mensagens de status da compilação
2. **Tabela de Símbolos** completa com todos os identificadores
3. Árvore Sintática Abstrata (AST) em formato textual
4. Mensagens de erro (se houver)

📖 Linguagem C-

Palavras-chave

- `if`, `else`, `while`, `return`
- `int`, `void`

Operadores

Aritméticos: `+`, `-`, `*`, `/`

Relacionais: `<`, `<=`, `>`, `>=`, `==`, `!=`

Atribuição: `=`

Delimitadores

- `;` (ponto-vírgula)
- `,` (vírgula)
- `()` (parênteses)
- `[]` (colchetes)
- `{}` (chaves)

Comentários

```c
/* Comentário em bloco */
```

💡 Exemplos de Código C-

Exemplo 1: Função Simples

```c
int soma(int a, int b) {
    return a + b;
}

void main(void) {
    int x;
    x = soma(5, 3);
}
```

Exemplo 2: Estruturas de Controle

```c
```

```
int fatorial(int n) {
    int resultado;
    resultado = 1;

    while (n > 1) {
        resultado = resultado * n;
        n = n - 1;
    }

    return resultado;
}
```

Exemplo 3: Arrays

```c
int soma_array(int arr[], int tam) {
    int i;
    int soma;

    soma = 0;
    i = 0;

    while (i < tam) {
        soma = soma + arr[i];
        i = i + 1;
    }

    return soma;
}
```

🔍 Mensagens de Erro

Erro Léxico

```
ERRO LEXICO: '@' - LINHA: 5
```

Erro Sintático

```
ERRO SINTATICO: token inesperado '}', esperado ';' - LINHA: 10
```

🧪 Testes

O projeto inclui arquivos de teste na pasta de exemplos. Para executar:

```bash
```

```
make test
```

🏗️ Estrutura da AST

A AST representa a estrutura hierárquica do programa, incluindo:

- **Declarações**: Variáveis e funções
- **Statements**: Comandos (if, while, return, atribuição)
- **Expressões**: Operações aritméticas e relacionais
- **Chamadas de Função**: Invocação de funções

🎯 Características Implementadas

- ☑ Reconhecimento de todos os tokens da linguagem C-
- ☑ Análise sintática completa
- ☑ Construção da AST
- ☑ **Tabela de Símbolos com gerenciamento de escopos**
- ☑ **Registro de variáveis, funções e parâmetros**
- ☑ **Suporte a tipos int e void**
- ☑ **Detecção de escopos (global e local)**
- ☑ Tratamento de erros léxicos
- ☑ Tratamento de erros sintáticos com linha
- ☑ Suporte a declarações de variáveis e arrays
- ☑ Suporte a declarações de funções
- ☑ Suporte a expressões aritméticas e relacionais
- ☑ Suporte a estruturas de controle (if, while)
- ☑ Suporte a chamadas de função
- ☑ Comentários em bloco

🔧 Limpeza

Para remover arquivos temporários e o executável:

```bash
make clean
```

📚 Referências

- LOUDEN, Kenneth C. **Compiler Construction: Principles and Practice**. Boston: PWS Publishing Company, 1997.
- Documentação do Flex: https://github.com/westes/flex
- Documentação do Bison: https://www.gnu.org/software/bison/

👥 Autores

Projeto desenvolvido para a disciplina de Compiladores - UNIFESP SJC

📄 Licença

Este projeto é desenvolvido para fins educacionais.